3 How to Record and Playback Movements

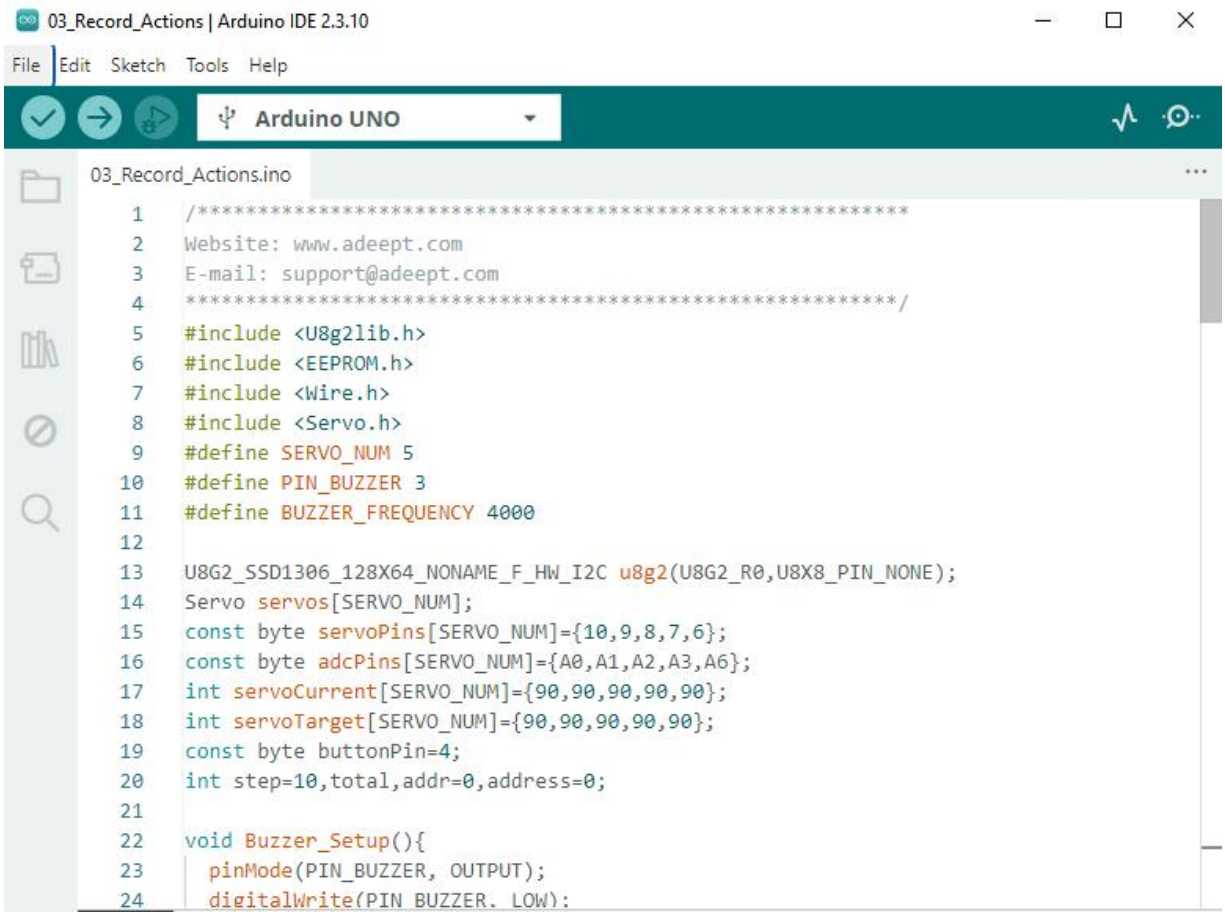
3.1 Overview

In this lesson, we will learn how to implement the motion recording function via EEPROM, enabling the robotic arm to automatically execute pre-recorded movements.

3.2 Upload Program

1. Connect your computer and Adeept Pixie Board with a USB cable.Then connect the battery and turn on the power switch.

2. Open "**03_Record_Actions**" folder in "**/Code**", double-click "**03_Record_Actions.ino**".



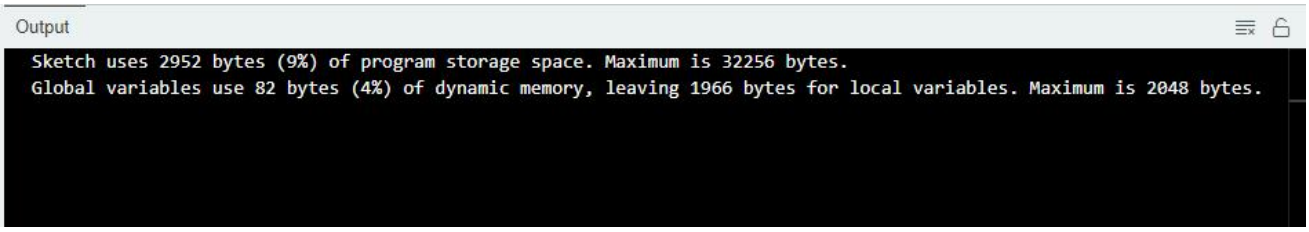
3. Select development board and serial port.

Board: **Tools--->Board--->Arduino AVR Boards--->Arduino Uno**

Port: **Tools --->Port--->COMx**

Note: The port number will be different in different computers.

4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.

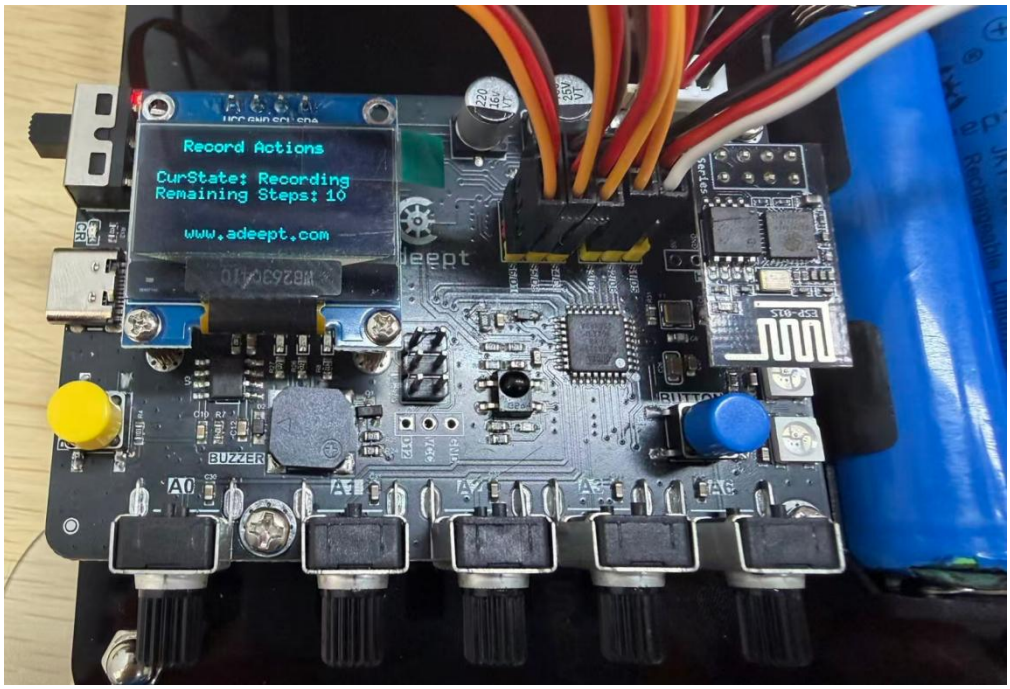


3.3 Introduction to Recording Function Operations

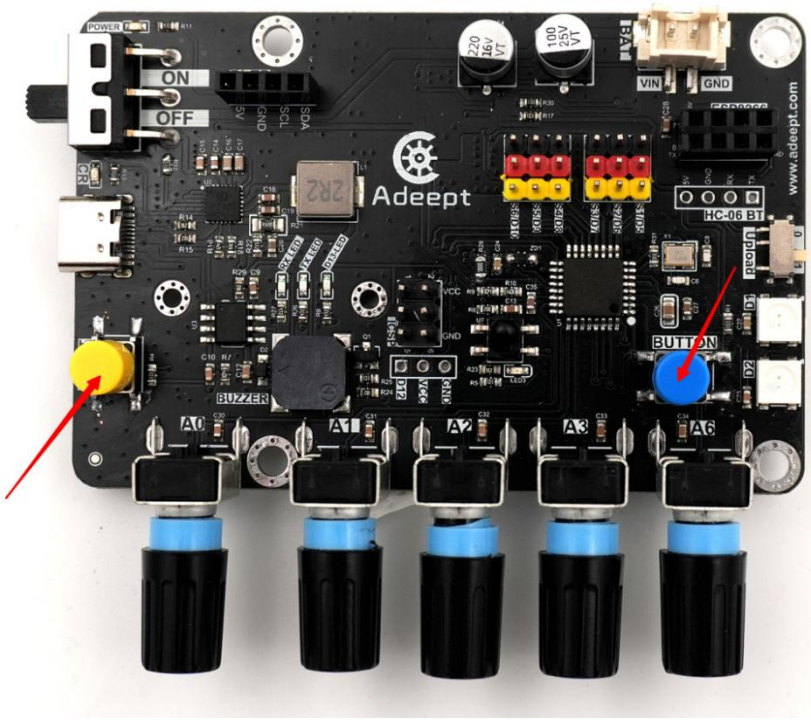
The recording function is implemented via EEPROM. You can record the servo angles of each robotic arm posture using buttons, and the data will be written to the storage chip. The recorded data will not be lost even after power off, and the previously recorded movements can still be executed when the device restarts.

The operation steps for this function are introduced as follows:

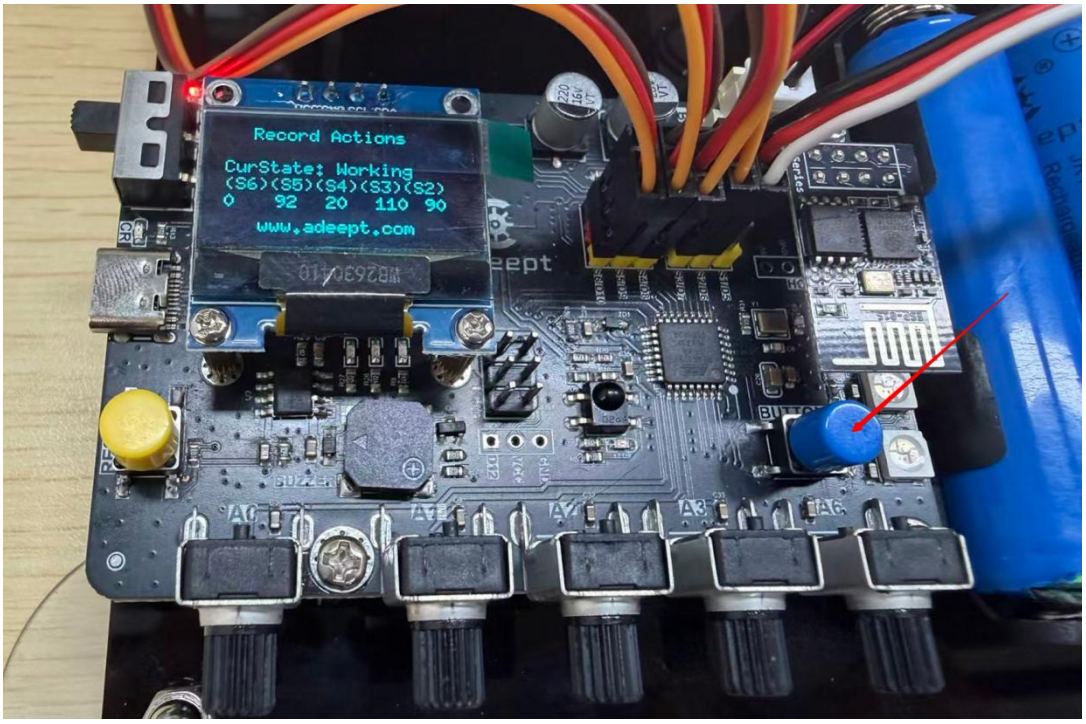
1.After powering on, you will hear a beep sound, which means the robotic arm has started successfully. At this time, the OLED screen will display that the current mode is recording mode, along with the remaining recordable steps (the default value is 10). You can modify the step count in the program and re-upload the code to apply changes.



2. Adjust the posture of the robotic arm via the potentiometer, then press the "**Button**" key on the control board to save the current posture. Repeat this process until the remaining recordable steps reach zero. The OLED screen will indicate that the device has switched to working mode, and the robotic arm will repeatedly execute the movements you have set.If you need to re-record movements, press the **RESET** button to restart the device.



3. After powering off, the recorded postures of the robotic arm are stored on the control board. The device will default to recording mode every time it boots up. To switch to working mode, hold down the "**Button**" key for **8** seconds, and the robotic arm will start executing the pre-recorded movements.



3.4 Code

```
/*************************************************************
Website: www.adeept.com
E-mail: support@adeept.com
*************************************************************/
#include <EEPROM.h>
#include <Servo.h>
#include <Wire.h>
#include <SSD1306AsciiWire.h>

#define SERVO_NUM 5
#define PIN_BUZZER 3
#define BUZZER_FREQUENCY 4000
#define OLED_I2C_ADDR   0x3C
#define PIN_BATTERY     A7

SSD1306AsciiWire oled;
const float ADC_REF_VOLTAGE    = 5.1;
const float VOLTAGE_DIV_RATIO  = 2.0;
const float BAT_MIN_VOLTAGE    = 6.0;
```

```
020 const float BAT_MAX_VOLTAGE    = 8.4;
021 float batteryVoltage = 0.0;
022
023 Servo servos[SERVO_NUM];
024 const byte servoPins[SERVO_NUM]={10,9,8,7,6};
025 const byte adcPins[SERVO_NUM]={A0,A1,A2,A3,A6};
026 int servoCurrent[SERVO_NUM]={90,90,90,90,90};
027 int servoTarget[SERVO_NUM]={90,90,90,90,90};
028 const byte buttonPin=4;
029 int step=10,total,addr=0,address=0;
030 unsigned long oledTime;
031
032 void setup(){
033   Serial.begin(115200);
034   for(int i=0;i<SERVO_NUM;i++) servos[i].attach(servoPins[i]);
035   pinMode(buttonPin,INPUT);
036   OLED_Setup();
037   servoMove();
038   total=step*SERVO_NUM;
039   Buzzer_Setup();
040   Buzzer_Alert(2,1);
041 }
042
043 void loop(){
044   int timer=0;
045   if(step){
046     for(int i=0;i<SERVO_NUM;i++)
047       servoTarget[i]=(i==SERVO_NUM-
048 1)?map(analogRead(adcPins[i]),0,1023,80,180):map(analogRead(adcPins[i]),0,1023,0,180);
049   }
050
051   servoMove();
052
053   if(digitalRead(buttonPin)==LOW){
054     while(digitalRead(buttonPin)==LOW){
055       delay(50);
056       if(timer<50) timer++;
057     }
058
059     if(timer==50){
060       step=0;
061       Buzzer_Alert(2,2);
062     }else if(step){
063       step--;
064       for(int i=0;i<SERVO_NUM;i++) EEPROM.update(addr+i,servoTarget[i]);
065       addr+=SERVO_NUM;
066       Buzzer_Alert(1,1);
067     }
068   }
069
070   if(step==0){
071     for(int i=0;i<SERVO_NUM;i++) servoTarget[i]=EEPROM.read(address+i);
072     address+=SERVO_NUM;
073     if(address>=total) address=0;
074   }
075
```

```
076   OLED_display(step);
077 }
078
079 void Buzzer_Setup(){
080   pinMode(PIN_BUZZER, OUTPUT);
081   digitalWrite(PIN_BUZZER, LOW);
082 }
083
084 void softTone(int durationMs){
085   unsigned long start = millis();
086   int delayUs = 500000 / BUZZER_FREQUENCY; // 半周期
087   while (millis() - start < durationMs) {
088     digitalWrite(PIN_BUZZER, HIGH);
089     delayMicroseconds(delayUs);
090     digitalWrite(PIN_BUZZER, LOW);
091     delayMicroseconds(delayUs);
092   }
093 }
094
095 void Buzzer_Alert(int beat, int rebeat){
096   beat = constrain(beat, 1, 9);
097   rebeat = constrain(rebeat, 1, 255);
098   for (int j = 0; j < rebeat; j++) {
099     for (int i = 0; i < beat; i++) {
100       softTone(100);
101       delay(100);
102     }
103     delay(500);
104   }
105 }
106
107 void servoMove(){
108   bool moving;
109   do{
110     moving=false;
111     for(int i=0;i<SERVO_NUM;i++){
112       if(servoCurrent[i]<servoTarget[i]){
113         servoCurrent[i]++;
114         moving=true;
115       }else if(servoCurrent[i]>servoTarget[i]){
116         servoCurrent[i]--;
117         moving=true;
118       }
119       servos[i].write(servoCurrent[i]);
120     }
121     if(moving) delay(10);
122   }while(moving);
123 }
124
125 void OLED_display(int step){
126   if(millis()-oledTime<1000) return;
127   oledTime=millis();
128
129   if(step==0){
130     oledPrintStr(60,3,"Working");
131     oledPrintStr(0,4,"                      ");
```

```
  }else{
    oledPrintStr(60,3,"Recording");
    oledPrintStr(0,4,"Remaining Steps:");
    oledPrintNum(100,4,step);
  }
  for(int i=0;i<SERVO_NUM;i++) oledPrintNum(5+i*25,7,servoTarget[i]);

  int batRatio = Get_Battery_Ratio();
  oledPrintNum(100, 1, batRatio);
  oledPrintStr(115, 1, "%");
}


void OLED_Setup(){
  Wire.begin();
  oled.begin(&Adafruit128x64,OLED_I2C_ADDR);
  oled.setFont(Adafruit5x7);
  oled.clear();

  oledPrintStr(15,0,"Record Actions");
  oledPrintStr(0,1,"Battery Percent:");
  oledPrintStr(0,3,"CurState:");
  oledPrintStr(0,6,"(S6)(S5)(S4)(S3)(S2)");
}

void oledPrintNum(int x,int y,int num){
  oled.set1X();
  oled.setCursor(x,y);
  oled.print(F("     "));
  oled.setCursor(x,y);
  oled.print(num);
}

void oledPrintStr(int x,int y,const char *text){
  oled.set1X();
  oled.setCursor(x,y);
  oled.print(F("                    "));
  oled.setCursor(x,y);
  oled.print(text);
}

int Get_Battery_Voltage_ADC(){
  int adcValue = 0;
  for (int i = 0; i < 5; i++){
    adcValue += analogRead(PIN_BATTERY);
    delayMicroseconds(200);
  }
  return adcValue / 5;
}

float Get_Battery_Voltage(){
  int adcValue = Get_Battery_Voltage_ADC();
  batteryVoltage = ((float)adcValue / 1023.0) * ADC_REF_VOLTAGE * VOLTAGE_DIV_RATIO;
  batteryVoltage = constrain(batteryVoltage, 0, BAT_MAX_VOLTAGE);
  return batteryVoltage;
}
```

```
188
189 int Get_Battery_Ratio(){
190   float voltage = Get_Battery_Voltage();
191   int ratio = map(voltage * 100, BAT_MIN_VOLTAGE * 100, BAT_MAX_VOLTAGE * 100, 0, 100);
192   return constrain(ratio, 0, 100);
    }
```